

1. Declaração da classe

O que significa?

- CounterBloc → sua classe responsável pela lógica do contador.
- extends → herda de outra classe.
- Bloc<CounterEvent, CounterState> → este BLoC recebe eventos do tipo CounterEvent e produz estados do tipo CounterState.

2. Construtor e super

O construtor CounterBloc() é executado quando criamos o BLoC.

super(CounterState(0)) chama o construtor da classe Bloc e define o estado inicial do contador como 0.

Sem isso, o BLoC não saberia qual estado usar ao iniciar.

3. Registrando o evento de incrementar

Passo a passo:

- on<CounterIncrementPressed> → escuta esse tipo de evento.
- event → o objeto do evento recebido (não é usado aqui).
- emit → função que envia um novo estado para a interface.
- state → estado atual do BLoC.
- state.count + 1 → soma 1 ao valor atual.
- emit(CounterState(...)) → cria e publica um novo estado com o valor atualizado.

Exemplo: estado atual = 5 → usuário aperta “+” → novo estado emitido = 6.

4. Registrando o evento de decrementar

Funciona exatamente da mesma forma, mas subtrai 1 do contador.

Exemplo: estado atual = 5 → usuário aperta “-” → novo estado emitido = 4.

O fluxo completo

1. Usuário toca no botão

2. Evento é enviado

CounterIncrementPressed ou CounterDecrementPressed

3. CounterBloc processa

O método on<...> correspondente é executado.

4. Novo estado é emitido

```
emit(CounterState(...))
```

5. A UI atualiza

Widgets como BlocBuilder recebem o novo estado e redesenham a tela automaticamente.

Resumo para memorizar

Código	Função
<code>extends Bloc<E,S></code>	Define eventos (E) e estados (S) do BLoC.
<code>super(CounterState(0))</code> <code>)</code>	Define o estado inicial do contador.
<code>on<Evento>(...)</code>	Registra um manipulador para um evento.
<code>state</code>	Representa o estado atual do BLoC.
<code>emit(...)</code>	Publica um novo estado para a aplicação.
<code>CounterState(...)</code>	Objeto imutável que guarda o valor do contador.

Uma frase para a prova: “O CounterBloc recebe eventos (incrementar/decrementar), calcula um novo valor a partir do estado atual (state) e o publica com emit; a interface escuta esse novo estado e se atualiza automaticamente